

3D Facial Thai Speech Animation Generation for Game Development

Phatthanit Musikaprayoon
 Department of Computer
 Engineering
 Faculty of Engineering
 Chulalongkorn University
 Bangkok, Thailand
 6470246121@student.chula.a
 c.th

Vishnu Kotrajaras
 Department of Computer
 Engineering
 Faculty of Engineering
 Chulalongkorn University
 Bangkok, Thailand
 vishnu@cp.eng.chula.ac.th

Abstract— This paper presents a method for generating 3D facial animations of characters speaking Thai, with the objective of augmenting realism in game production. The main issue is the lack of tools and datasets specifically designed for Thai phonetic features, which differ from other languages in terms of sound structure and articulation. To resolve this issue, we created a plugin for Unreal Engine that uses phonetic data to generate realistic face motions, using the TSynC-3miti as the main sources for model training. With particular attention to the phonetic needs of the Thai language, facial motions are produced by combining a sliding window approach for temporal sequence prediction with an ensemble decision tree model. The plugin takes Thai text as input, converts it into phonetic symbols, and generates synchronized face motions for a MetaHuman 3D character model in Unreal Engine. Testing showed that our approach generates more realistic animations than Motion capture and Audio driven method, whether viewed from the front or at an angle. Our model delivers a more precise representation of pronunciation across different phonetic symbols. This capability makes it well-suited for broader applications in interactive media and customized content designed for Thai-speaking audiences.

Keywords— *Speech animation, 3D Facial Animation, Game development, Machine Learning*

I. INTRODUCTION

Realistic character animations in games and virtual worlds have become more popular in recent years, especially as localized content and interactive media that target certain language groups have become more mainstream. Realistic facial movements that seamlessly align with in-game character's spoken language enhance immersion and create a more lifelike interactive experience. Creating animations that accurately reflect speech presents significant challenges, especially when adapting to languages like Thai, which features distinctive phonetic characteristics.

The Thai language differs significantly from English and other languages in terms of sound articulation, phonetic structure, and tonal diversity. Although deep learning models and animation creation tools are getting better in creating speech animations, these techniques are often based on English or multilingual datasets that inadequately support Thai phonetics.

As a result, animations produced by these models often exhibit lip motions that lack the realism needed for authentic Thai speech, diminishing the overall credibility of the character's performance.

This study introduces a method for generating 3D facial animations tailored specifically for Thai-language speech, overcoming existing limitations. The approach aims to enhance the accuracy of lip movements and pronunciation alignment, improving realism in speech animations for Thai-speaking users. We developed a model capable of predicting face movements that correspond to Thai phonetic sequences. Our datasets consisted of TSynC-3miti, which included audiovisual data of Thai speech. The model was integrated into a MetaHuman 3D character [7] with an Unreal Engine 5 [8] plugin, enabling fluid speech animations straight from Thai text input. Our methodology employed an ensemble decision tree model alongside a sliding window strategy for spatiotemporal sequence prediction, facilitating precise and efficient animation synthesis for on-demand application.

This study's goals are to: (1) design a system for producing 3D face animations of Thai speech; (2) predict phonetic-driven facial motion using decision tree ensemble model with sliding window techniques; and (3) assess the animations' realism and applicability for game development. We expect that the realistic face animations produced by this model will play a significant role in the advancement of Thai-language content in interactive media and gaming.

II. RELATED WORK

In recent years, research in speech animation has come a long way, thanks in large part to data-driven and deep learning approaches. However, most of the previous research focused on English or other widely spoken languages, resulting in an underrepresentation of languages such as Thai, both in the areas of dedicated datasets and speech animation models.

A. Thai Language Audiovisual Dataset

The development of speech-driven facial animation systems in Thai has been constrained by the limited availability of annotated audiovisual datasets that capture the specific phonetic and prosodic characteristics of the language. The majority of

current corpora have been created for English or other widely spoken languages and do not effectively generalize to Thai because of its tonal characteristics and syllabic structure.

To address these problems, Thangthai et al. [1] presented TSynC-3miti corpus, a large Thai audiovisual speech synthesis dataset derived from existing media. Unlike traditional studio based datasets, TSynC-3miti was compiled from over 19 hours of televised news broadcasts, focusing on a single professional speaker to guarantee phonetic precision. It includes synchronized audio, phoneme-level Thai transcriptions, aligned facial landmarks, and facial action units (AUs) extracted using OpenFace. Additionally, it facilitates retargeting to 3D character rigs by converting information to ARKit-compatible blendshapes. In addition, Kasuriya et al. [17] presented the Thai emotional speech corpus from Lakorn (EMOLA). However, due to copyright limitations, this corpus could not be used in our work.

The dataset minimizes major weaknesses of previous Thai corpora small size, high recording costs, and lack of facial annotations by using a semi-automated system for data cleansing, phonetic alignment (utilizing Montreal Forced Aligner), and audiovisual synchronization. TSynC-3miti is a valuable resource for training speech animation models specifically designed for Thai, facilitating both linguistic accuracy and facial motion synthesis.

B. Speech Animation Models

Several machine learning methodologies have been suggested to generate face animation from speech [14], focusing on various input sources like audio waveforms, phoneme sequences, and linguistic features. These methodologies aim to capture complex articulatory behaviors, including coarticulation and prosodic variation, which are essential for realistic lip synchronization.

Focusing on lip animation, several studies have yielded remarkable results. Taylor et al. [2] suggested a sliding window based deep learning model that predicts sequences of facial motion features from input phoneme labels. Their method uses overlapping windows of phoneme sequences to capture localized temporal curvature in the motion signal and visual coarticulation effects. It is made to work with a variety of CG characters in real time. It is, to some extent, able to generalize between speakers and languages. The model prioritizes simplicity, retargetability, and integration with production workflows.

Kim et al. [3] created a decision tree framework for spatiotemporal sequence prediction, focusing on high dimensional, continuous output spaces like face motion features. This method divides sequence prediction into a series of fixed-length overlapping windows, with each subsequence being predicted using a regression model with multiple variables tree. The method supports fast training and prediction, and handles imperfect training data (e.g., missing or misaligned motion labels) through a latent variable extension. This flexibility is especially useful in audiovisual datasets obtained from non-studio sources, where getting precise alignment is difficult. In experimental evaluations, their methodology (Spatiotemporal Sliding Window with Decision Trees) overcame traditional

sequence learning algorithms such as SEARN and DAGger, and either equaled or exceeded the efficacy of HMMbased models and dynamic visemes.

Complementary to decision tree-based models, Karras et al. [4] proposed an end-to-end neural architecture that maps raw audio input to 3D facial mesh deformations. Their framework continuously learns position and a latent emotional representation, allowing dynamic and expressive speech-driven animation. The framework includes a formant analysis network and uses convolutional layers with fully connected outputs for controlling over 5,000 facial vertex in real-time.

For research in Thai speech facial animation, Chuensaichol et al. [16] presented the traditional computer graphics and animation approach. Laphawan et al. [15] presented an intriguing alternative approach utilizing VocalTractLab. However, there is currently no existing work that maps VocalTractLab to a commercial-grade 3D face model. Moreover, the proposed model still does not include consonants and a predefined syllable segmentation of the input is required.

By leveraging machine learning, we can enhance the accuracy and efficiency of phoneme-to-viseme mapping, reduce manual effort, and potentially achieve more realistic and dynamic speech-driven facial animations.

Although these methodologies have demonstrated strong performance in English and other widely studied languages, they require adjustments when applied to tonal languages such as Thai. This is due not only to the presence of tonal variation but also to unique phonetic elements—such as the Thai consonant “ng” sound which does not occur at the beginning of words in English. In this study, we integrated the decision-tree-based spatiotemporal modeling approach of Kim et al., including the phoneme-aligned, fixed size sliding window technique also employed by Taylor et al., and adopt the system to accommodate Thai phonetic characteristics using the TSynC-3miti dataset. This hybrid solution enabled the on-demand synthesis of facial motion in Unreal Engine’s MetaHuman framework and supported accurate modeling of tonal contours and syllabic distinctions specific to Thai speech.

III. APPROACH

For generating facial animation while speaking Thai, we used a sequence of phonetic symbols as input. The challenge was the lack of a specialized dataset that captures the phonetic intricacies of Thai. We preprocessed the TSynC-3miti audiovisual dataset to produce the required input. Subsequently, we trained a model to forecast the associated face action and provided a tool for integrating the model into production processes. The process was as follows

A. Data Preparation

The data preparation steps for training the model using the TSynC-3miti dataset are as follows:

1) *Tokenizing Words*: Tokenization was performed using the PyThaiNLP [5] library, which provides tools for Thai natural language processing. The method `pythainlp.tokenize` was used with the “deepcut” engine.

2) *Extracting Audio*: To align phonetic symbols with audio, the speech must be extracted from the video. The moviepy library was used for audio extraction from video files.

3) *Extracting Facial Data*: Facial data was extracted using OpenFace [6], which processes a video to detect facial movements. The output was a CSV file containing frame data, timestamp, gaze, head pose, facial landmarks, and facial action units (AU). Since the MetaHuman 3D model in Unreal Engine included built-in functions to control facial movements, it was possible to use this model directly by importing facial movement data in the Facial Action Coding System (FACS) [9] format in facial action units (AU). As the main goal of this research was to move the mouth for speech, only AUs that deal with mouth control were used.

4) *Aligning Phonetic Symbols*: The phonetic sequence was synchronized with the audio data with the Montreal Forced Aligner (MFA) [10]. The mfa_train_and_align method analyzed audio and produced a TextGrid file containing phonetic symbols with corresponding timestamps, organized into layers that represent the speech sequence.

5) *Preprocessing and Sliding Window Technique for Phonetic Data*: The following steps were taken to handle the phonetic data after receiving the TextGrid file:

a) *Align the phonetic sequence with the facial movement data based on timestamps.*

b) *Eliminate tone markers from the phonetic symbols, as they do not influence mouth shape during articulation. Fig. 1 shows the tone markers for the mid tone, low tone, falling tone, high tone and rising tone in Thai.*

┆ : Mid tone
┆ : Low tone
V : Falling tone
┆ : High tone
A : Rising tone

Fig. 1. Tone markers in Thai.

c) *Transform the phonetic data into numerical sequences for model training.*

d) *Apply min/max scaling to normalize the data.*

e) *Apply a sliding window technique to systematically arrange the data.*

6) *Preprocessing and Sliding Window Technique for Facial Data*: The following is the pre-processing of the face movement data:

a) *Apply the sliding window technique to arrange the data.*

b) *Calculate the average movement for each window in the facial action units.*

c) *Apply min/max scaling to normalize the data.*

B. Development of the Model for Predicting 3D Facial Movements and Training the Model with the Dataset

The model developed for this project was based on the framework outlined by Kim et al. [3]. TSynC-3miti was used as the training dataset. It was pre-processed and arranged as previously described. The model predicts a series of face movements, aligned with the corresponding speech input.

Experimenting with the ensemble decision tree model and adjusting the parameters of each model, including Gradientboosted trees, Random forests, AdaBoost, and XGBoost, the Gradient-boosted trees model, using the GradientBoostingRegressor function in sklearn with parameters $n_estimators=200$, $learning_rate=0.1$, and $max_depth=3$, produced the best results, as assessed by the Mean Squared Error and R-squared score. The input window size was set at 7 ($K_x=7$), and the output window size before averaging was set to 5 ($K_y=5$). Fig. 2 shows an example of using sliding window technique with the model.

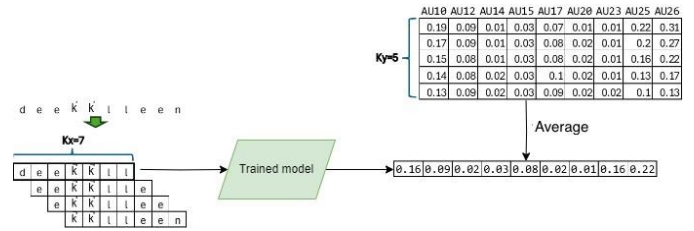


Fig. 2. Example of using sliding window technique with the model.

C. Development of Plugin for Animation Creation

This study used the developed model to create a plugin for Unreal Engine that generated 3D MetaHuman speech animation from text input.

This plugin converted text into a series of phonetic symbols, and employed this sequence to obtain facial movement values using the trained model. The predicted facial movement values were subsequently aligned with the reference points of the character's face at each time interval, resulting in a 3D animation of the MetaHuman character model. The detailed working of the tool is as follows:

1) *Tokenization*: For text tokenization, the PyThaiNLP tool was used. The pythainlp.tokenize function was used with the parameter engine set to "deepcut".

2) *Text-to-Speech (TTS) Generation*: For generating speech audio from text, the VAJA9 [11] system, an open API created by the National Electronics and Computer Technology Center (NECTEC), was used. A WAV file that corresponded to the provided text was obtained.

3) *Phonetic Alignment*: The next step was to create a phonetic sequence from audio and text token for each time interval using the Montreal Forced Aligner (MFA). The mfa_train_and_align function was used to generate a TextGrid file with phonetic timestamps.

4) *Preprocessing and Sliding Window Technique for Phonetic Data*: After acquiring the TextGrid file with the

phonetic sequence and timings, the phonetic data was subjected to preprocessing. This included the following steps:

a) *Elimination of tone marks, as they had no effect on lip movements in speaking.*

b) *Conversion of phonetic data into a numerical sequence.*

c) *Min-max scaling was applied according to the parameters used in the training dataset.*

d) *Applying a sliding window technique for data formatting.*

5) *Facial Movements:* The trained model in the previous phase predicted the facial movement sequence. A sliding window of phonetic symbols served as input to the trained model.

6) *Postprocessing:* After obtaining the facial movement predictions for each action unit (related to lip control), postprocessing is performed to adjust the animation. This may involve smoothing, blending, or additional refinement to ensure the animation is fluid and realistic.

a) *Using a Rule-Based Algorithm for Phonetics with Clear Lip Movements.*

- For the phonetics shown in Fig. 3, the algorithm adjusted the lip movement to show a lip closure. Our program adjusted all action units (AU) from these phonetics to 0.

b	: u
m	: m
p	: p
p ^h	: f, w, g
p [~]	: u, p, f, w, g (final consonants)

Fig. 3. Lip closure phonetics.

- For the phonetics shown in Fig. 4, the algorithm adjusted the lip movement to show lip rounding. Our program increased the action unit AU14 by 0.3.
- For the phonetics shown in Fig. 5, the algorithm adjusted the lip movement to show small lip rounding. Our program increased the action unit AU14 by 0.1.

o	: โอะ
o:	: โอ
u	: อุ
u:	: อู
ua	: อัวะ, อัว
w	: ว

Fig. 4. Lip rounding phonetics.

๑	: เอะ
๑:	: ออ

Fig. 5. Small lip rounding phonetics.

b) *Smoothing the Values:* Due to the nature of the model's predictions, when applied to the character, there might be irregular movements. Smoothing was given to each action unit to enhance the realism of lip motions. The smoothed values of each action unit were computed using a moving average with a window size of 3, based on the data at each time point. Subsequently, backfilling was used to replace any NaN values. This produced a smoothed data for every time point. The calculation for each time point was executed using the following equation:

$$Smoothed_t = \frac{x_{t-1} + x_t + x_{t+1}}{3} \quad (1)$$

Where x_t represents the value of an action unit at time t .

Example: Given the data = [1,2,3,4,5]

- Calculate Moving Average with a Window Size of 3:

$$Smoothed = \begin{bmatrix} Smoothed(1,2,3) \\ Smoothed(2,3,4) \\ Smoothed(3,4,5) \end{bmatrix} = [2, 3, 4] \quad (2)$$

- After combining the original values with two NaNs:

$$Smoothed = [NaN, NaN, 2, 3, 4] \quad (3)$$

- Fill missing values (NaNs) using backfilling method:

$$Smoothed = [2, 2, 2, 3, 4] \quad (4)$$

c) *Min-max scaling was applied according to the parameters used in the training dataset.*

d) *Applying a sliding window technique for data formatting.*

7) *Retargeting:* Table I shows the mapping of movement values to the MetaHuman character. A script was written to import movement value into the MetaHuman character. Fig. 6 contains reference points on the face corresponding to time. When the animation is played, the character model will move according to the provided values.

TABLE I. ACTION UNIT WITH METAHUMAN CONTROLLER MAPPING

OpenFace parameters	Control Mouth/Jaw	Left	Right	Up	Down
AU10	Upper Lip Raise	✓	✓		
AU12	Corner Pull	✓	✓		
	Sharp Corner	✓	✓		
	Pull Corner	✓	✓		
AU14	Lips Press	✓	✓	✓	✓
	Lip Sticky	✓	✓		
	Push Pull	✓	✓	✓	✓
	Towards	✓	✓	✓	✓
	Purse	✓	✓	✓	✓
AU15	Corner Depress	✓	✓		
	Corner Sharpness	✓	✓	✓	✓
AU17	Chin Compress	✓	✓		
	Funnel	✓	✓	✓	✓
AU20	Stretch	✓	✓		
	Stretch Lips Close	✓	✓		
AU23	Tighten	✓	✓	✓	✓
AU25	Lower Lip Depress	✓	✓		
AU26	Jaw				✓

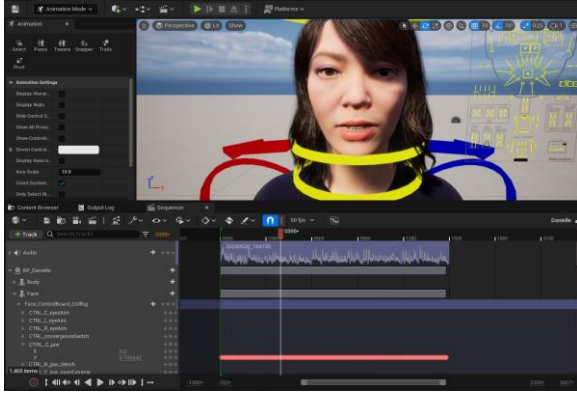


Fig. 6. Retargeting controls (bottom left).

IV. EVALUATION

The results of our 3D facial animation of the mouth while speaking Thai were tested and evaluated in order to determine how realistic the animation is.

1) Method for Testing the Realism of the Animation

a) Two test articles containing all consonants, vowels, and final sounds was constructed in the Thai language. Some short vowel sounds were omitted to avoid making the sentences too long. Tests used long vowels with similar sounds instead. The two articles are shown in Table. II.

TABLE II. ACTION UNIT WITH METAHUMAN CONTROLLER MAPPING

Article no.	Sentence no.	Thai	Phonetics
1	1	เมื่อวานที่พ่อพาแม่ไป ขับกับเพื่อนได้มีโอกาสไปเยือน สวนสัตว์ที่ใหญ่ที่สุดในประเทศ	mua/wa:n/tʰu:ŋ/fa:/tsemsaj/tɕʰan/kāp/pʰuan/da:j/mi:/tɔ:kə:t/paj/juan/suan/sāt/tʰi:/ja/tʰi:sūt/naj/pratʰe:t
	2	การเดินทางมาแสดงจากสวนนานาชาติ ทั้งสัตว์บก และสัตว์น้ำ เช่นช้าง เสือ ค่างขาว และนก ไก่เรียนรู เกี่ยวกับสัตว์หายาก	ka:n/dy:n/tɕʰom/ka:n/sade:ŋ/tɕʰa:k/sāt/na:/tɕʰanit/tʰan/sāt/bok/le/sāt/na:m/tɕʰen/tɕʰa:ŋ/sua/kʰa:ŋw/le/nok/da:j/rianu:/kiaw/kāp/sāt/ha:/ja:k
	3	ทำให้ง่วงนอนมากและยังได้ทานอาหารร่วมกัน อย่างอร่อย	tʰam/haj/ru:suŋk/tw:ne/ma:k/le/jaŋ/da:j/tʰa:n/ʔa:hn/ruam/kan/ja:ŋ/ʔarətɕj
2	1	การเดินทางไปท่องเที่ยวเขาในอุทยานนั้น สนุกสนานมาก	ka:n/dy:n/tʰa:ŋ/paj/tʰuŋgiaw/pʰu:kʰaw/naj/ru:du:/na:w/na:sanuka:/ma:k
	2	โดยเฉพาะการเดินทางด้วยรถไฟขบวนพิเศษ เป็นพิเศษ	do:j/tɕʰapʰu:ʔ/ka:n/dy:n/tʰa:ŋ/duaj/rōtɕaj/tɕʰom/tʰi:wāt/tʰi:/ŋōta:m/pen/pʰise:t
	3	ทำให้ง่วงนอนและผ่อนคลายร่างกายและน้ำอัดใจ	tʰam/haj/mi:/kʰwa:m/suŋk/le/ʔ/pʰuŋkʰa:j/ra:ŋk/ban/tɕitaj
	4	เมื่อถึงยอดเขาแล้วก็นั่งพักผ่อนกับวิวเมืองด้านล่าง	mua/tʰuŋ/ʔu:t/kʰaw/le:w/ko:/maj/lu:m/tʰi:/tɕa:prɔ:t/hu:/pɔ:t/ta:/kāp/wimuan/da:n/la:ŋ

b) Using the two articles as input, three animation models were compared: Ground Truth, Karras et al.'s approach, and our proposed method. The generated animations for each model were displayed in both the frontal and side views (tilted face), as shown in Fig 7 and Fig 8. The animations were recorded in video format.



Fig. 7. Front view face.



Fig. 8. Side view face.

Each Ground Truth animation was generated using the Live Link Face application [12], which captured facial motion and streamed it onto a MetaHuman character. For the Karras et al. approach, we generated an audio file from the article and imported it into NVIDIA Omniverse Audio2Face [13]. The resulting animation from Audio2Face was then mapped onto a MetaHuman character. Our approach followed the steps outlined previously.

c) *Questionnaire Design*: The questionnaire collected general information about the evaluator, including age range, gender, occupation, experience in animation creation or design, familiarity with AI tools for speech processing (e.g., Deepfake, Text-to-Speech, Speech-to-Text, Voice Generators), and experience playing games with lip-syncing characters (e.g., GTA V, Sims 4). Each animation was segmented into multiple smaller parts, after which evaluators were asked to judge the realism of each part by ranking it among the three approaches. Additional realism evaluation was carried out by having 31 participants grade our animation of the two test articles, using the scale of 1 to 5.

2) Analysis of Realism Evaluation

Out of 130 evaluators, 94 were aged 35 or younger, and 36 were over 35 years old. Among the evaluators, 29 had experience in creating animations, while 101 did not. Additionally, 69 evaluators had experience using generative AI, and 61 did not. Lastly, 96 evaluators had played 3D games, while 34 had no such experience. Table III shows the list of generated animations, together with the approach that was ranked first by evaluators for each animation. The animations number 1-3 are portions from Article 1 in the angled view, while number 4-7 are from Article 2 in the same angled view. Number 8-10 show the animations from Article 1 in the frontal view, and number 11-14 are from Article 2 in the frontal view. Among all the animations, both the frontal and angled views of our model performed the best in most sentences, except for number 6 and 13, where the same sentence appears in both views. In these cases, Karras et al.'s model was more realistic, as our model exhibited more jerky movements based on feedback received. It was found that phonetic symbols, which were postprocessed by a rule-based algorithm, occurred too frequently in succession, resulting in the animation appearing jerky. Overall, the general

characteristics of the evaluators (age, animation creation experience, generative AI experience, game development experience and 3D game playing experience) did not significantly affect the results. The number of people who selected 1st rank for each animation portion is shown in Fig. 9.

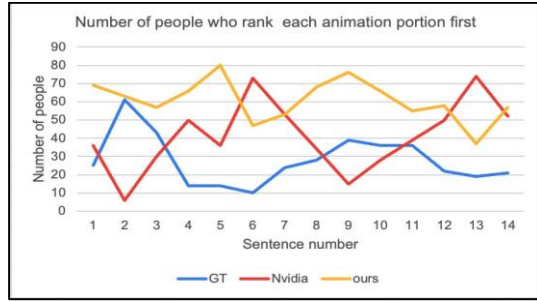


Fig. 9. The number of participants voting 1st rank for each animation sentence.

TABLE III. EVALUATION RESULT

No.	Face Angle	Article no.	Sentence no.	Approach ranked 1st by evaluators
1	45	1	1	ours
2	45	1	2	ours
3	45	1	3	ours
4	45	2	1	ours
5	45	2	2	ours
6	45	2	3	Karras's
7	45	2	4	Karras's/ours
8	90	1	1	ours
9	90	1	2	ours
10	90	1	3	ours
11	90	2	1	ours
12	90	2	2	ours
13	90	2	3	Karras's
14	90	2	4	ours

The ground truth displayed smooth movements overall, though it struggled with minimal mouth opening and occasional misalignment with the audio, particularly during softer speech. The Karras et al. model from NVIDIA Omniverse Audio2Face showed accurate lip sync in some segments but was often stiff and jerky, with mismatched timing and excessive mouth openings. Our model was the best in terms of accuracy, but it still exhibited unnatural movements, particularly when postprocessing was done too frequently. For the additional 1 to 5 scale realism evaluation, provided by 31 participants, the average rating for the front-facing view of the character was 3.98 (76.56%), with a standard deviation of 0.86. For the angled view, the average score was 4.13 (82.68%). Participants were satisfied with the realism quality of our animation, according to Customer Satisfaction Score (CSAT) [18].

3) Analysis of Realism Evaluation for Each Phonetic Symbol

The realism evaluation for all phonetic symbols was relatively consistent, except for the “aw” vowel sound. This symbol was rated by participants as the least realistic when

compared to other phonetic symbols, both in the frontal and angled views of the character. Based on the author’s analysis, the sound involves a slight rounding of the lips, which is less pronounced than for the vowels “o” and “u”.

V. TOOL USAGE

To use the tool, the following steps were carried out:

1) Prepare the computer for testing with the following specifications:

- Computer: OS: Microsoft Windows 11 Pro, Processor: AMD Ryzen 9 5900X 12-Core Processor, 3701 MHz, 12 Core(s), 24 Logical Processor(s), Memory (RAM): 32 GB, GPU: NVIDIA GeForce RTX 3080
- Program: Python: Python 3.11.5, TensorFlow: 2.13.0, Montreal-Forced-Aligner: 3.0.0a4, Unreal Engine: 5.1

2) Install the GenSpeech plugin (Ours) into the sample project.

3) Prepare the MetaHuman character and call the blueprint function BP_ProcessAnimMNG from the GenSpeech plugin.

4) Configure the character by specifying the MetaHuman character name in the blueprint function BP_ProcessAnimMNG, as shown in Fig. 10

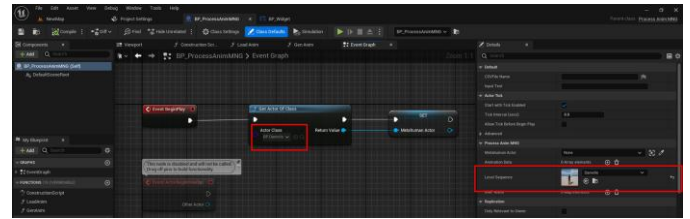


Fig. 10. Setting up GenSpeech plugin.

5) Activate the service for generating movement values in Python’s Virtual Environment.

6) Test by entering text into the GUI to send data to the service for generating movement values, and wait for the animation result as shown in Fig. 11. The sentences (50-60 words) took about 3-10 seconds to process.

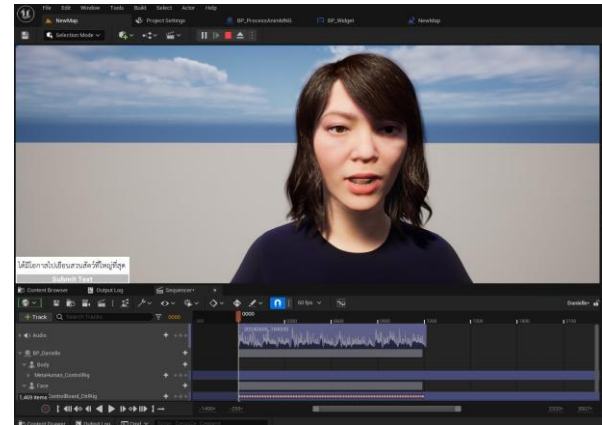


Fig. 11. Text entry (left) and animation face.

With the use of the proposed plugin, games will be able to generate speech animations on-demand during runtime. When a user submits text input, its corresponding Thai speech animation will be displayed within a few seconds.

VI. CONCLUSION

This paper introduces a novel method for generating 3D facial animations of Thai-speaking characters. We developed a plugin for Unreal Engine that employs phonetic data, integrating a sliding window methodology with an ensemble decision tree model to provide synchronized facial animations for MetaHuman 3D character models. Our methodology surpasses current techniques, such as Motion Capture and Audio driven approach. Despite some issues with jerky movements in certain cases, our method demonstrates a precise representation of Thai pronunciation and provides a promising solution for interactive media and content tailored for Thai-speaking users.

Future work could focus on refining the postprocessing steps to enhance animation fluidity and extend the system's applicability to other tonal languages. Additionally, the system could be expanded to incorporate other facial elements and support the generation of emotional expressions, further improving the realism and expressiveness of the animations.

REFERENCES

- [1] A. Thangthai, S. Thatphithakkul, K. Thangthai and A. Namsanit, "TSynC-3miti: Audiovisual Speech Synthesis Database from Found Data," 2020 23rd Conference of the Oriental COCOSDA International Committee for the Co-ordination and Standardisation of Speech Databases and Assessment Techniques (O-COCOSDA), Yangon, Myanmar, 2020, pp. 77-82, doi: 10.1109/O-COCOSDA50338.2020.9295001.
- [2] S. Taylor, T. Kim, Y. Yue, M. Mahler, J. Krahe, A. G. Rodriguez, J. Hodgins and I. Matthews, "A deep learning approach for generalized speech animation," *ACM Transactions On Graphics (TOG)*, vol. 36, pp. 1-11, 2017.
- [3] T. Kim, Y. Yue, S. Taylor and I. Matthews, "A Decision Tree Framework for Spatiotemporal Sequence Prediction," *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, p. 577-586, 2015.
- [4] T. Karras, T. Aila, S. Laine, A. Herva and J. Lehtinen, "Audio-driven facial animation by joint end-to-end learning of pose and emotion," *ACM Trans. Graph.*, vol. 36, 2017.
- [5] W. Phatthiyaphaibun, K. Chaovavanich, C. Polpanumas, A. Suriyawongkul, L. Lowphansirikul and P. Chormai, "PyThaiNLP: Thai Natural Language Processing in Python," 2024. [Online]. Available: <https://github.com/PyThaiNLP/pythainlp/>.
- [6] T. Baltrusaitis, A. Zadeh, Y. C. Lim and L. P. Morency, "OpenFace 2.0: Facial Behavior Analysis Toolkit," in 2018 13th IEEE International Conference on Automatic Face & Gesture Recognition (FG 2018), 2018.
- [7] "Unreal Engine: MetaHuman," [Online]. Available: <https://www.unrealengine.com/en-US/metahuman>. [Accessed 22 April 2025].
- [8] "Unreal Engine," [Online]. Available: <https://www.unrealengine.com/enUS>. [Accessed 22 April 2025].
- [9] "Facial Action Coding System," [Online]. Available: <https://www.paulekman.com/facial-action-coding-system/>. [Accessed May 2024].
- [10] "Montreal Forced Aligner documentation," [Online]. Available: <https://montreal-forced-aligner.readthedocs.io/en/latest/>. [Accessed June 2024].
- [11] "Vaja9," [Online]. Available: <https://ai4orthai.in.th/services/vaja9.html>. [Accessed June 2024].
- [12] "Facial Capture with Live Link," [Online]. Available: <https://dev.epicgames.com/community/learning/tutorials/IEYE/unrealengine-facial-capture-with-live-link> [Accessed 22 April 2025].
- [13] "Omniverse Audio2Face," [Online]. Available: <https://docs.omniverse.nvidia.com/audio2face/latest/index.html> [Accessed 22 April 2025].
- [14] D. Jiang, J. Chang, L. You, S. Bian, R. Kosk, and G. Maguire, "AudioDriven Facial Animation with Deep Learning: A Survey," *Information*, vol. 15, no. 11, Art. no. 675, 2024, doi: 10.3390/info15110675.
- [15] T. Laphawan, S. Prom-On, P. Birkholz and Y. Xu, "Estimating Underlying Articulatory Targets of Thai Vowels by Using Deep Learning Based on Generating Synthetic Samples From a 3D Vocal Tract Model and Data Augmentation," in *IEEE Access*, vol. 10, pp. 41489-41502, 2022, doi: 10.1109/ACCESS.2022.3166922.
- [16] T. Chuensaichol, P. Kanongchaiyos and C. Wutiwiwatchai, "Thai Speech-Driven Facial Animation," 2011 Second International Conference on Culture and Computing, Kyoto, Japan, 2011, pp. 121-122, doi: 10.1109/Culture-Computing.2011.30.
- [17] S. Kasuriya, T. Theeramunkong, C. Wutiwiwatchai, and P. Sukhummeek, "Developing a Thai emotional speech corpus from Lakorn (EMOLA)," *Language Resources and Evaluation*, vol. 53, no. 1, pp. 17-55, Mar. 2019, doi: 10.1007/s10579-018-9428-9.
- [18] G. Raileanu, "CSAT: Definition, Calculation & 2024 Benchmarks," **Retently**, Jun. 2024. [Online]. Available: <https://www.retently.com/blog/customer-satisfaction-score-csat/>. [Accessed: Apr. 26, 2025].